

Name: Kishan Patel

Reg No:25MML030

Lab Work (19 Feb 2026)

Aim: To extract frames from a video and analyze them using a pre-trained MobileNetV2 model for image recognition and basic edge/video intelligence processing.

Dataset: LIST

Applications:

1. **Smart Surveillance Systems** – Detect and recognize objects or activities in video streams.
2. **Autonomous Vehicles** – Identify objects like pedestrians, vehicles, and road signs.
3. **Traffic Monitoring** – Analyze vehicles and traffic flow from video footage.
4. **Smart Cameras / Edge Devices** – Perform real-time video analysis without relying heavily on cloud processing.
5. **Video Content Analysis** – Automatically classify or recognize objects in video frames.

Code:

```
import cv2
import numpy as np
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input, decode_predictions

# Load pretrained MobileNetV2
model = MobileNetV2(weights='imagenet')

import cv2
import numpy as np

def extract_frames(video_path, max_frames=20):
    cap = cv2.VideoCapture(video_path)

    if not cap.isOpened():
        print("Error: Cannot open video")
        return np.array([])

    frames = []
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    # Handle videos where frame count is 0
    if total_frames > 0:
        step = max(1, total_frames // max_frames)
    else:
        step = 1

    count = 0
```

```

while True:
    ret, frame = cap.read()
    if not ret:
        break

    if count % step == 0:
        frame = cv2.resize(frame, (224, 224))
        frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB) # Important fix
        frames.append(frame)

    count += 1

    if len(frames) >= max_frames:
        break

cap.release()
print("Extracted frames:", len(frames))
return np.array(frames)

```

```

def classify_video(video_path):
    frames = extract_frames(video_path)

    if len(frames) == 0:
        print("No frames extracted!")
        return

    frames = preprocess_input(frames)
    predictions = model.predict(frames)

    # Average predictions across frames
    avg_pred = np.mean(predictions, axis=0)
    #top_pred = decode_predictions(np.expand_dims(avg_pred, axis=0), top=1)

    top_pred = decode_predictions(np.expand_dims(avg_pred, axis=0), top=5)

    for pred in top_pred[0]:
        print(pred[1], ":", round(pred[2]*100, 2), "%")

    print("Final Prediction:", top_pred[0][0][1])

# Replace with your video path
video_path = "real_world_demo_video.mp4"
classify_video(video_path)

```

```

... Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_224.h5
14536120/14536120 0s 0us/step
Extracted frames: 20
1/1 14s 14s/step
Downloading data from https://storage.googleapis.com/download.tensorflow.org/data/imagenet_class_index.json
35363/35363 0s 0us/step
digital_clock : 63.84 %
traffic_light : 9.13 %
switch : 1.43 %
computer_keyboard : 1.32 %
matchstick : 0.85 %
Final Prediction: digital_clock

```

Explanation:

This code uses the MobileNetV2 deep learning model to classify objects in a video. It first extracts up to 20 frames from the video, resizes them to the required size (224×224), and preprocesses them. The frames are then passed to the pretrained MobileNetV2 model, which predicts objects in the frames, and the top predicted classes are displayed with their confidence percentages.

```

import cv2
import numpy as np
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input, decode_predictions

# Load pretrained MobileNetV2
model = MobileNetV2(weights='imagenet')

def extract_frames(video_path, max_frames=20):
    cap = cv2.VideoCapture(video_path)

    if not cap.isOpened():
        print("Error: Cannot open video")
        return np.array([])

    frames = []
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    # step size for uniform sampling
    if total_frames > 0:
        step = max(1, total_frames // max_frames)
    else:
        step = 1

    count = 0

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        if count % step == 0:
            frame = cv2.resize(frame, (224, 224))
            frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            frames.append(frame)

        count += 1

```



```
        if len(frames) >= max_frames:
            break

    cap.release()
    print("Extracted frames:", len(frames))
    return np.array(frames)

def show_first_frame(frames):
    if len(frames) == 0:
        print("No frames to display")
        return

    first_frame = frames[0]

    # Convert RGB → BGR (OpenCV needs BGR to display correctly)
    first_frame_bgr = cv2.cvtColor(first_frame, cv2.COLOR_RGB2BGR)

    cv2.imshow("First Frame", first_frame_bgr)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```



```
def classify_video(video_path):
    frames = extract_frames(video_path)

    if len(frames) == 0:
        print("No frames extracted!")
        return

    # ===== DEBUG PRINTS =====
    print("\n===== FRAME INFORMATION =====")

    # shape of all frames
    print("All frames shape:", frames.shape)

    # shape of single frame
    print("Single frame shape:", frames[0])

    # one pixel value
    print("One pixel value frames[0][0][0]:", frames[0][0][0])

    # first row first 5 pixels
    print("First 5 pixels of first row:\n", frames[0][0][:5])

    # datatype
    print("Data type:", frames.dtype)
    plt.imshow(frames[0])

    print("===== \n")
```


Explanation:

This code extracts **frames from a video**, resizes and preprocesses them, and then uses the **pretrained MobileNetV2 deep learning model** to classify the objects present in the video. It also prints **frame information (shape, pixel values, datatype)** and displays the first frame before showing the **top predicted object classes with their confidence percentages**.

```
1 # ----- FRAME EXTRACTION -----
2 def extract_frames(video_path, max_frames=20):
3     cap = cv2.VideoCapture(video_path)
4
5     if not cap.isOpened():
6         print("Error: Cannot open video")
7         return np.array([])
8
9     frames = []
10    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
11
12    step = max(1, total_frames // max_frames) if total_frames > 0 else 1
13
14    count = 0
15    while True:
16        ret, frame = cap.read()
17        if not ret:
18            break
19
20        if count % step == 0:
21            frame = cv2.resize(frame, (224, 224))
22            frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
23            frames.append(frame)
24
25        count += 1
26        if len(frames) >= max_frames:
27            break
28
29    cap.release()
30    return np.array(frames)
31
32 # ----- DISPLAY + MATRIX -----
33 def display_frames_and_values(frames):
34
35     if len(frames) == 0:
36         print("No frames extracted")
37         return
38
39     print("Total Frames:", len(frames))
40     print("Frames Shape:", frames.shape)
41     print("\n===== MATRIX VALUES =====\n")
42
43     # Print matrix values for each frame
44     for i in range(len(frames)):
45         print(f"\nFrame {i+1} matrix values:\n")
46         print(frames[i])      # entire pixel matrix
47         print("-----")
48
49     # Show all frames in grid
50     cols = 5
51     rows = int(np.ceil(len(frames) / cols))
52
53     plt.figure(figsize=(15, 8))
54
55     for i in range(len(frames)):
56         plt.subplot(rows, cols, i+1)
57         plt.imshow(frames[i])
58         plt.title(f"Frame {i+1}")
59         plt.axis('off')
60
61     plt.tight_layout()
62     plt.show()
```

```
# ----- MAIN -----
video_path = "real_world_demo_video.mp4"
```

```
frames = extract_frames(video_path)
display_frames_and_values(frames)
```

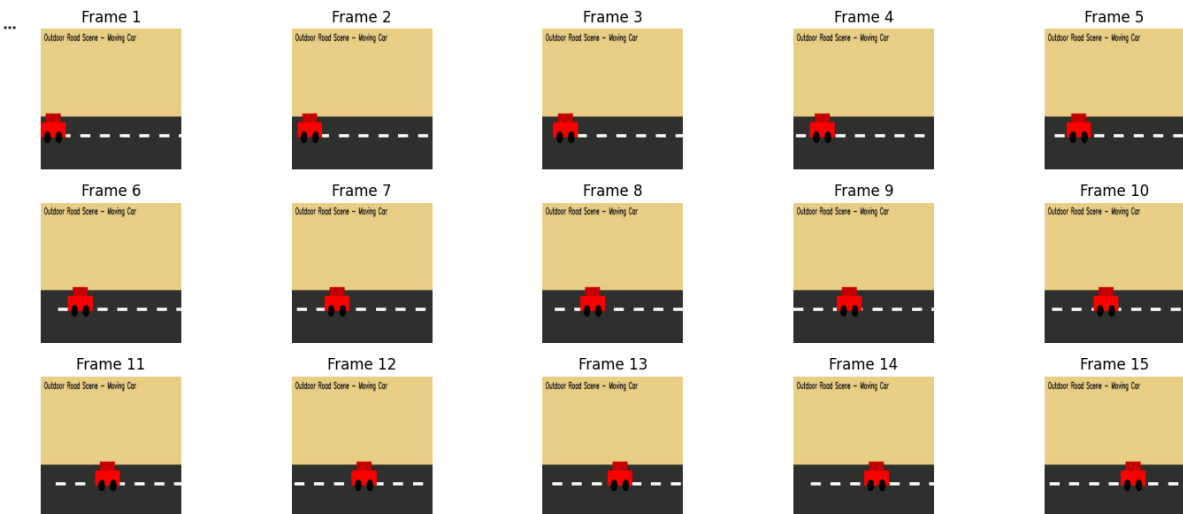
```
*** ===== MATRIX VALUES =====
```

```
Frame 1 matrix values:
```

```
[[[233 206 133]
  [233 206 133]
  [233 206 133]
  ...
  [233 206 133]
  [233 206 133]
  [233 206 133]]
```

```
[[[233 206 133]
  [233 206 133]
  [233 206 133]
  ...
  [233 206 133]
  [233 206 133]
  [233 206 133]]
```

```
[[[233 206 133]
  [233 206 133]
  [233 206 133]]
```



Explanation:

This code extracts **up to 20 frames from a video**, resizes them to **224×224**, and converts them to **RGB format**. It then **prints the pixel matrix values of each frame** and displays all the

Challenges with Solutions

1. **Large video size and high computation**

Solution: Extracted only **20 frames** from the video instead of processing all frames.

2. **Different frame sizes in video**

Solution: Resized all frames to **224 × 224** for consistent processing.

3. **Incorrect color representation (BGR vs RGB)**

Solution: Converted frames from **BGR to RGB** using `cv2.cvtColor()`.

4. **Handling videos with unknown frame counts**

Solution: Used a **step size method** to uniformly sample frames from the video.

5. **Difficulty visualizing frame data**

Solution: Displayed frames in a **Matplotlib grid** and **printed pixel matrix values** for analysis.

Lab Assessment(5th Feb 2026)

Aim: To process and analyze video frames using **OpenCV in Python**, applying techniques like **grayscale conversion, edge detection, and frame extraction** to understand basic computer vision and edge video processing.

Applications:

1. **Smart Surveillance Systems** – Used to detect motion, identify objects, and monitor activities in security cameras.
2. **Autonomous Vehicles** – Helps vehicles detect road edges, obstacles, and surrounding objects using computer vision.
3. **Traffic Monitoring** – Used to analyze vehicle movement and manage traffic flow.
4. **Smart Cameras and IoT Devices** – Processes video data at the edge to reduce cloud processing and improve real-time decision making.
5. **Video Analytics Systems** – Used in applications like crowd monitoring, gesture detection, and activity recognition.

Code:

```
from google.colab import drive
drive.mount('/content/drive')
```

... Mounted at /content/drive

```
from IPython.display import display, Javascript
from google.colab.output import eval_js
from base64 import b64decode
import cv2
import numpy as np
```

```

def take_photo(filename='photo.jpg', quality=0.8):
    js = Javascript('''
        async function takePhoto(quality) {
            const div = document.createElement('div');
            const capture = document.createElement('button');
            capture.textContent = 'Capture';
            div.appendChild(capture);

            const video = document.createElement('video');
            video.style.display = 'block';
            const stream = await navigator.mediaDevices.getUserMedia({video: true});

            document.body.appendChild(div);
            div.appendChild(video);
            video.srcObject = stream;
            await video.play();

            google.colab.output.setIframeHeight(document.documentElement.scrollHeight, true);

            await new Promise((resolve) => capture.onclick = resolve);

            const canvas = document.createElement('canvas');
            canvas.width = video.videoWidth;
            canvas.height = video.videoHeight;
            canvas.getContext('2d').drawImage(video, 0, 0);

            stream.getVideoTracks()[0].stop();
            div.remove();

            return canvas.toDataURL('image/jpeg', quality);
        }
    ''')
    display(js)
    data = eval_js('takePhoto({})'.format(quality))
    binary = b64decode(data.split(',')[1])

    return canvas.toDataURL('image/jpeg', quality);
    ''')
    display(js)
    data = eval_js('takePhoto({})'.format(quality))
    binary = b64decode(data.split(',')[1])

    with open(filename, 'wb') as f:
        f.write(binary)

    return filename

```

```
take_photo()
```

```
'photo.jpg'
```

Explanation of the Code

This code defines a function **take_photo()** that captures an image using the **webcam in Google Colab** and saves it as a JPEG file.

1. The function creates a **JavaScript interface** that accesses the user's webcam through the browser and displays a **video stream with a "Capture" button**.
2. When the user clicks the **Capture button**, the current frame from the webcam video is drawn onto a **canvas element** and converted into a JPEG image.
3. The captured image is then **encoded in base64 format**, sent back to Python, decoded into binary data, and saved as a file (default name **photo.jpg**).

In summary: this function allows the notebook to **capture a photo from the webcam and store it as an image file for further processing**.

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

def extract_grayscale_frame(video_path, frame_number=0):
    # 1. Load the video
    cap = cv2.VideoCapture(video_path)

    # Set the frame position
    cap.set(cv2.CAP_PROP_POS_FRAMES, frame_number)
    success, frame = cap.read()

    if not success:
        print("Error: Could not read frame.")
        return None

    # 2. Convert to Grayscale
    gray_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # 3. Calculate Metrics
    mean_val = np.mean(gray_frame)
    std_val = np.std(gray_frame)

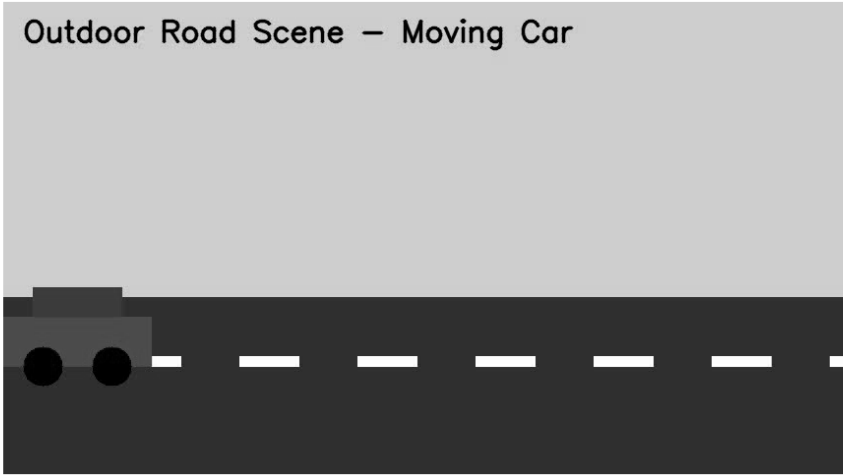
    print(f"Metrics for Frame {frame_number}:")
    print(f"- Mean Intensity: {mean_val:.2f}")
    print(f"- Standard Deviation: {std_val:.2f}")

    # Display result in Colab
    cv2_imshow(gray_frame)

    cap.release()
    return gray_frame

# Usage
path = '/content/drive/MyDrive/Edge_Intelligence/real_world_demo_video.mp4'
gray_img = extract_grayscale_frame(path, frame_number=0)
```

```
*** Metrics for Frame 0:
- Mean Intensity: 146.15
- Standard Deviation: 77.75
```



Explanation:

This code loads a video, extracts a specific frame, and converts it into a **grayscale image** using OpenCV. It then calculates the **mean and standard deviation of pixel intensity** and displays the grayscale frame in Google Colab.

```

import cv2
import numpy as np
from google.colab.patches import cv2_imshow

def process_video_frame(video_path, frame_number=0):
    cap = cv2.VideoCapture(video_path)
    cap.set(cv2.CAP_PROP_POS_FRAMES, frame_number)
    success, frame = cap.read()
    cap.release()

    if not success:
        return "Error: Could not read video."

    # Convert to Grayscale
    gray_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # 1. Show Image Metrics
    print(f"--- Frame {frame_number} Metrics ---")
    print(f"Dimensions: {gray_frame.shape}")
    print(f"Mean Intensity: {np.mean(gray_frame):.2f}")
    print(f"Min/Max: {np.min(gray_frame)} / {np.max(gray_frame)}")

    # 2. Show the Matrix (Top-left 10x10 slice)
    print("\n--- Pixel Matrix (10x10 Slice) ---")
    print(gray_frame[:10, :10]) # Rows 0-9, Columns 0-9

    # 3. Show the Visual
    print("\n--- Visual Output ---")
    cv2_imshow(gray_frame)

# Usage
path = '/content/drive/MyDrive/Edge_Intelligence/real_world_demo_video.mp4'
process_video_frame(path)

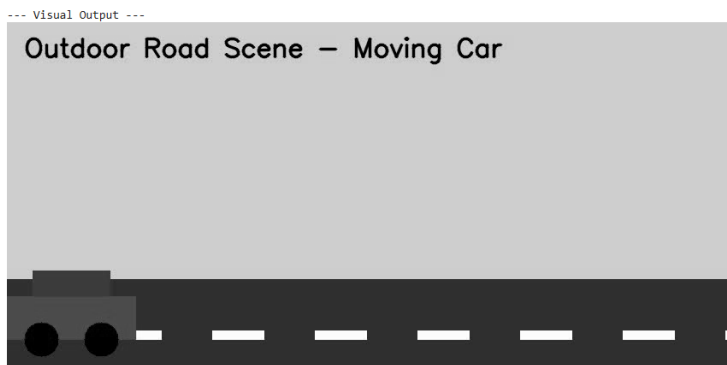
```

```

--- Frame 0 Metrics ---
Dimensions: (480, 854)
Mean Intensity: 146.15
Min/Max: 0 / 255

--- Pixel Matrix (10x10 Slice) ---
[[206 206 206 206 206 206 206 206 206 206]
 [206 206 206 206 206 206 206 206 206 206]
 [206 206 206 206 206 206 206 206 206 206]
 [206 206 206 206 206 206 206 206 206 206]
 [206 206 206 206 206 206 206 206 206 206]
 [206 206 206 206 206 206 206 206 206 206]
 [206 206 206 206 206 206 206 206 206 206]
 [206 206 206 206 206 206 206 206 206 206]
 [206 206 206 206 206 206 206 206 206 206]
 [206 206 206 206 206 206 206 206 206 206]]

```



Explanation:

This code extracts a **specific frame from a video**, converts it to **grayscale**, and displays information about the image such as **dimensions, mean intensity, and minimum/maximum pixel values**. It also prints a **10×10 pixel matrix sample** and shows the grayscale frame visually in Google Colab.

```

import cv2
import numpy as np
from google.colab.patches import cv2_imshow

def divide_and_edge_detect(video_path, sample_rate=30):
    cap = cv2.VideoCapture(video_path)
    frame_count = 0

    if not cap.isOpened():
        print("Error: Video not found.")
        return

    print(f"--- Processing Video: {video_path} ---")

    while True:
        success, frame = cap.read()
        if not success:
            break # Stop when video ends

        # 1. Grayscale Conversion
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        # 2. Edge Detection (Canny)
        # 100 and 200 are the threshold values for edge intensity
        edges = cv2.Canny(gray, 100, 200)

        # 3. Display every 'sample_rate' frame to avoid crashing the browser
        if frame_count % sample_rate == 0:
            print(f"\n--- Frame {frame_count} Edge Map ---")
            print(f"Edge Pixel Matrix (Top 5x5):\n{edges[:5, :5]}")
            cv2_imshow(edges)

        frame_count += 1

    cap.release()
    print(f"\nTotal frames processed: {frame_count}")

    print(f"\nVideo frames processed: {frame_count} /")

# Usage: sample_rate=30 shows 1 frame per second (assuming 30fps video)
path = '/content/drive/MyDrive/Edge_Intelligence/real_world_demo_video.mp4'
divide_and_edge_detect(path, sample_rate=50)

...
[[0 0 0 0]
 [0 0 0 0]
 [0 0 0 0]
 [0 0 0 0]
 [0 0 0 0]]

Outdoor Road Scene - Moving Car

--- Frame 50 Edge Map ---
Edge Pixel Matrix (Top 5x5):
[[0 0 0 0]
 [0 0 0 0]
 [0 0 0 0]
 [0 0 0 0]
 [0 0 0 0]]

```

Explanation:

This code reads a video and processes its frames by converting them to **grayscale** and applying **Canny edge detection** to highlight object boundaries. It displays the **edge-detected frame** and a **small pixel matrix sample** at a specific interval (`sample_rate`) to avoid displaying too many frames.

```

import cv2
import numpy as np
from google.colab.patches import cv2_imshow # Special function for displaying images in Google Colab
import time

# Define the path to your video file
# You may need to upload a file to your Colab environment or mount Google Drive
video_path = '/content/drive/MyDrive/Edge_Intelligence/real_world_demo_video.mp4'

# Create a VideoCapture object to read the video from the file path
cap = cv2.VideoCapture(video_path)

# Check if the video file was opened successfully
if not cap.isOpened():
    print("Error: Could not open video file.")
else:
    print("Video file opened successfully. Starting processing...")

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    ret, frame = cap.read()

    # If the frame was read correctly, ret is True
    if ret:
        # Convert the color frame to grayscale using cv2.cvtColor()
        gray_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        # *** FIND THE NEGATIVE OF THE IMAGE ***
        # Method 1: Using cv2.bitwise_not() (recommended for simplicity)
        negative_frame = cv2.bitwise_not(gray_frame) #

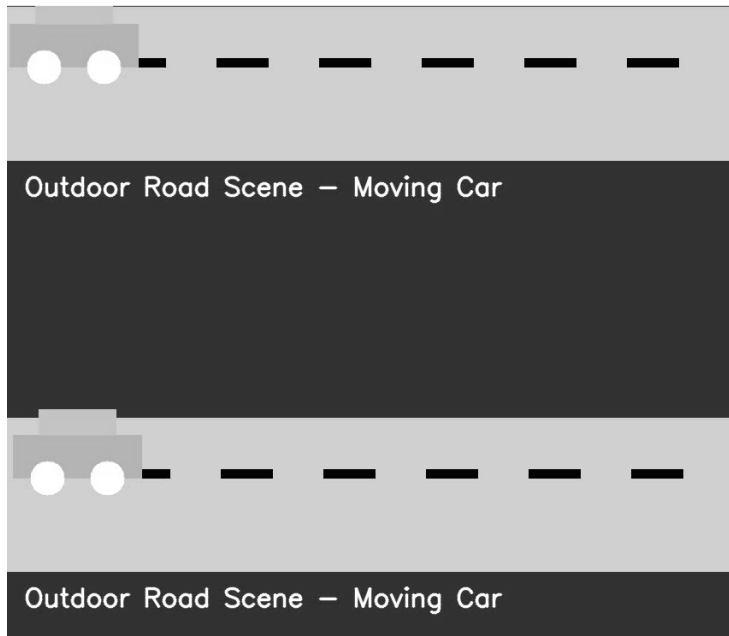
        # Method 2: Using array subtraction (as images are NumPy arrays)
        # negative_frame = 255 - gray_frame #

        # Display the negative frame in Google Colab
        cv2_imshow(negative_frame)

        # Optional: Add a small delay to control playback speed
        # Adjust the sleep time (in seconds) to achieve the desired frame rate
        time.sleep(0.03)
    else:
        # Break the loop if the video ends or an error occurs
        print("End of video or error reading frame.")
        break

# Release the video capture object and close any open windows
cap.release()
# cv2.destroyAllWindows() # Not necessary for cv2_imshow in Colab

```



Explanation:

This code reads a video frame by frame, converts each frame to **grayscale**, and creates the **negative of the image** using `cv2.bitwise_not()`. The negative frames are then displayed continuously in Google Colab to show the processed video output.

```

# Initialize the first frame for comparison
ret, prev_frame = cap.read()
if ret:
    prev_gray = cv2.cvtColor(prev_frame, cv2.COLOR_BGR2GRAY)
    prev_gray = cv2.GaussianBlur(prev_gray, (21, 21), 0) # Smooth to reduce noise

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    # 1. Pre-process current frame
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    gray = cv2.GaussianBlur(gray, (21, 21), 0)

    # 2. Calculate the Difference (Motion)
    # This subtracts the previous frame from the current one
    frame_delta = cv2.absdiff(prev_gray, gray)

    # 3. Thresholding
    # If the change is > 25, turn pixel white (motion), else black (still)
    thresh = cv2.threshold(frame_delta, 25, 255, cv2.THRESH_BINARY)[1]

    # 4. Dilate the threshold to fill in holes
    thresh = cv2.dilate(thresh, None, iterations=2)

    # Display the result
    print("--- Motion Mask ---")
    cv2.imshow(thresh)

    # Update the previous frame for the next iteration
    prev_gray = gray

    # Control playback speed
    time.sleep(0.01)

cap.release()

```



Explanation:

This code performs **motion detection in a video** by comparing the current frame with the previous frame. It calculates the **difference between frames**, applies thresholding to highlight moving areas, and displays a **motion mask showing where movement occurs in the video**.

Challenges:

1. High computational cost when processing video frames in real time.
2. Noise and lighting variations in frames can affect edge detection and motion detection accuracy.
3. Large video data size requires more memory and processing power.
4. Frame processing delay can occur when handling high-resolution videos.
5. Difficulty in detecting small or slow movements accurately.

Solutions Implemented:

1. Converted frames to grayscale to reduce computational complexity and speed up processing.
2. Processed frames at intervals (sample rate) instead of every frame to reduce workload.
3. Applied Gaussian blur to remove noise and improve motion detection accuracy.
4. Used edge detection and thresholding to highlight important features like edges and motion.
5. Displayed limited frames to prevent browser crashes and manage processing efficiently.